

SCORPIO: Serving Right Requests at the Right Time for Heterogeneous SLOs in LLM Inference

Yinghao Tang¹, Tingfeng Lan², Bo Pan¹, Xiuqi Huang^{1*}, Hui Lu^{3*}, and Wei Chen¹

¹ State Key Lab of CAD&CG, Zhejiang University

² Department of Computer Science, University of Virginia

³ Department of Computer Science and Engineering, The University of Texas at Arlington

*Corresponding authors: huangxiuqi@zju.edu.cn (primary), hui.lu@uta.edu

Abstract. Large Language Model (LLM) serving increasingly underpins online Web services such as conversational agents, Web search, and programming assistants, where requests carry heterogeneous Service Level Objectives (SLOs) such as Time to First Token (TTFT) and Time Per Output Token (TPOT). Existing LLM serving systems prioritize maximum throughput and treat all requests uniformly, which leads to suboptimal SLO attainment. This paper introduces SCORPIO, an SLO-oriented LLM serving system designed to maximize system goodput and SLO attainment for workloads with heterogeneous SLOs. Our core insight is to exploit SLO heterogeneity for adaptive scheduling across admission control, queue management, and batch selection. SCORPIO features a TTFT Guard, which employs least-deadline-first reordering and rejects unattainable requests, and a TPOT Guard, which utilizes a VBS-based admission control and a novel credit-based batching mechanism. Both guards are supported by a predictive module. Evaluations demonstrate that SCORPIO improves system goodput by up to $14.4\times$ and SLO adherence by up to 46.5% under high load compared to state-of-the-art baselines.

Keywords: LLM serving · Model-as-a-Service · Service Level Objectives · Request scheduling · Web service quality.

1 Introduction

Large Language Models (LLMs) have become core infrastructure for Web-delivered AI services, served to end users over the Web through Model-as-a-Service (MaaS) platforms that power conversational agents [14], Web search and deep-search engines [29], and programming assistants [16]. As these services scale to millions of concurrent Web requests, the inference backend behind each HTTP endpoint becomes the central performance bottleneck for user-facing quality. To handle the substantial computational requirements of LLM inference, state-of-the-art serving systems such as vLLM [9] and SGLang [34] employ advanced techniques, including continuous batching [30], paged attention [9], chunked

prefilling [1], etc. These optimizations markedly improve inference throughput and resource utilization.

Despite these efficiency improvements, existing LLM serving systems [9,30,34,17] predominantly prioritize maximum throughput, often neglecting Service Level Objectives (SLOs), such as time-to-first-token (TTFT) and time-per-output-token (TPOT). Typically, these systems greedily admit and serve incoming requests [15], without deep visibility into the SLO requirements of requests. Simultaneously, SLO requirements across different applications are inherently heterogeneous [19]. For instance, programming assistants often demand low latency for real-time response, whereas chatbots tolerate higher latency provided the generation rate exceeds human reading speed [12]. However, existing LLM serving systems treat all requests equally in all scheduling stages. This undifferentiated handling leads to suboptimal SLO attainment. These problems necessitate a fine-grained and adaptive scheduling method to support heterogeneous SLOs.

In this paper, we focus on designing an SLO-oriented LLM serving system that is tailored for heterogeneous SLOs, with the goal of maximizing both system goodput and SLO attainment. **Our key insight is that SLO heterogeneity is not a constraint to tolerate but a resource to exploit: by scheduling the right request at the right time across queue management, admission control, and batch selection, the system converts slack in loose-SLO requests into headroom for tight-SLO requests, lifting system-level SLO attainment without extra hardware.** From the TTFT perspective, requests with looser SLO can be served later. From the TPOT perspective, requests with looser TPOT can skip some iterations of generation, leaving more resources for requests with tighter TPOT.

Based on this insight, we design TPOT Guard and TTFT Guard to handle the heterogeneous TPOT and TTFT SLOs, respectively. For the TPOT Guard, we first define a core concept of TPOT-relative Proportionality (TRP), which quantifies the heterogeneity of TPOT SLOs. Utilizing TRP, we propose a novel VBS-based Admission Control mechanism and a Credit-based Batching mechanism. The former is to control the admission of requests to prevent mass TPOT violations due to overwhelming request ingress. The latter mechanism adaptively batches requests according to their per-request TPOT SLO, achieving system-level high TPOT SLO attainment. For the TTFT Guard, we implement a simple but effective least-deadline-first reordering strategy that prioritizes requests nearing their TTFT deadlines. Additionally, under heavy load, we reject the requests that are unattainable for their TTFT SLOs. To provide decision-making support for these two modules, we develop a predictor module consisting of a sequence length predictor and two analytical models. By orchestrating these complementary mechanisms in concert, SCORPIO enables robust support for diverse workloads.

Our contributions are summarized as follows:

- **New setting.** We co-optimize *heterogeneous* TTFT and TPOT SLOs within a single serving system, a setting that underpins multi-tenant Web services

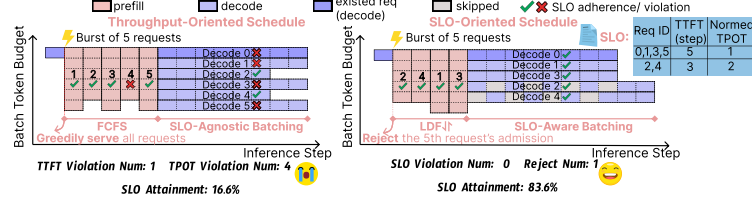


Fig. 1: Comparison of throughput-oriented and SLO-oriented scheduling approaches. The throughput-oriented scheduler greedily admits and processes requests without considering per-request SLOs. During the prefill phase, request 4 violates its SLO constraint (4 steps > 3 steps). The SLO-oriented scheduler prevents such violations through least-deadline-first prioritization. During the decode phase, we assume a normalized decode step time as $0.25 \times \text{BatchSize}$. The throughput-oriented scheduler batches all requests in each step ($\text{BatchSize}=6$), causing requests 0, 1, 3, and 5 to violate their TPOT constraints (each step consumes a time of 1.5). In contrast, the SLO-oriented scheduler rejects unattainable requests (e.g., request 5) and implements an adaptive fine-grained batching strategy ($\text{BatchSize}=4$). This strategy allows requests with looser TPOT SLOs (requests 2 and 4) to skip certain iterations, ensuring all admitted requests satisfy their TPOT constraints.

such as chatbots, coding assistants, and search, where different applications demand sharply different latency guarantees.

- **TPOT Guard.** We design a TPOT Guard built on a novel TPOT-relative Proportionality (TRP) abstraction, pairing Virtual-Batch-Size admission control with credit-based batching to drive per-request TPOT adherence under load.
- **TTFT Guard.** We design a TTFT Guard combining least-deadline-first re-ordering with proactive rejection of unattainable requests, preventing starvation while honoring heterogeneous TTFT deadlines.
- **Predictive module.** We build a lightweight predictive module, namely a fine-grained sequence-length predictor and two analytical latency models that estimate TPOT and TTFT with R^2 above 0.9, at under 1% scheduling overhead.
- **System and results.** We implement SCORPIO on vLLM and evaluate it on Llama-3.1-8B and Gemma-2-27B across ShareGPT, LMSYS-Chat-1M, and real Azure production traces. SCORPIO improves system goodput by up to **14.4×** and SLO adherence by up to **46.5%** over state-of-the-art baselines, with negligible (under 1%) scheduling overhead.

2 Background and Problem Formulation

A typical LLM generative inference task has two stages: i) the prefill stage, which takes a prompt sequence to generate the first output token; and ii) the decoding

stage, which generates new tokens autoregressively. The quality of LLM service is typically evaluated by two key metrics: *time to first token* (TTFT) and *time per output token* (TPOT) [35]. TTFT captures latency for generating the first output token after a request is received, while TPOT sets an upper bound on the average latency for generating subsequent tokens. Meeting SLOs plays a key role in providing high-quality LLM services and has been fully researched in other fields like cloud and edge computing [13,31]. However, this problem has not been well explored in LLM serving. Also, from the perspective of Model as a Service (MaaS) providers, different applications and users have different SLO requirements [19,3]. In a Web setting, these per-request SLOs are the latency contracts a MaaS endpoint exposes to downstream Web applications, so meeting them heterogeneously lets a single shared backend honor the diverse quality-of-service expectations of distinct Web tenants [7]. This inspires us to explore the heterogeneous SLO attainment problem in LLM serving.

We define the heterogeneous SLO attainment problem as follows: During a time interval T , there is a sequence of user requests $\mathcal{R} = \{r_1, r_2, \dots, r_N\}$ arriving. An LLM inference system processes the requests with a scheduling policy π . A request $r_i \in \mathcal{R}$ with its TTFT SLO threshold $S_{TT}(r_i)$ and TPOT SLO threshold $S_{TP}(r_i)$ is considered SLO-compliant if it satisfies both $TTFT(r_i) \leq S_{TT}(r_i)$ and $TPOT(r_i) \leq S_{TP}(r_i)$. Let $\mathcal{R}^{good}(\pi) \subseteq \mathcal{R}$ be the subset of those requests that are SLO-compliant. Note that the denominator $|\mathcal{R}|$ counts all arriving requests, so rejected requests are counted as SLO-unmet and never inflate either metric. Then the system goodput and SLO adherence rate can be defined as:

$$\text{Goodput}(\pi) = \frac{|\mathcal{R}^{good}(\pi)|}{T} \quad (1)$$

$$\text{Adherence}(\pi) = \frac{|\mathcal{R}^{good}(\pi)|}{|\mathcal{R}|} \quad (2)$$

The objective is to design an online scheduling policy π aiming to achieve:

$$\max_{\pi} \left(\lim_{T \rightarrow \infty} \mathbb{E}[\text{Goodput}(\pi, T)], \lim_{T \rightarrow \infty} \mathbb{E}[\text{Adherence}(\pi, T)] \right) \quad (3)$$

3 Method

3.1 System Overview

SCORPIO is a system-algorithm co-design framework designed to maximize goodput and SLO adherence. SCORPIO consists of three key components: 1) a predictor, 2) a TTFT Guard, and 3) a TPOT Guard. The two guards leverage predicted SLO metrics to explicitly handle heterogeneous request requirements.

Figure 3 shows the workflow of SCORPIO: when a request arrives, the predictor (§3.2) first predicts the output sequence length, which is used by two analytic models to estimate the TPOT and TTFT. Given the estimated information, the TTFT and TPOT Guards make scheduling decisions accordingly. Specifically,

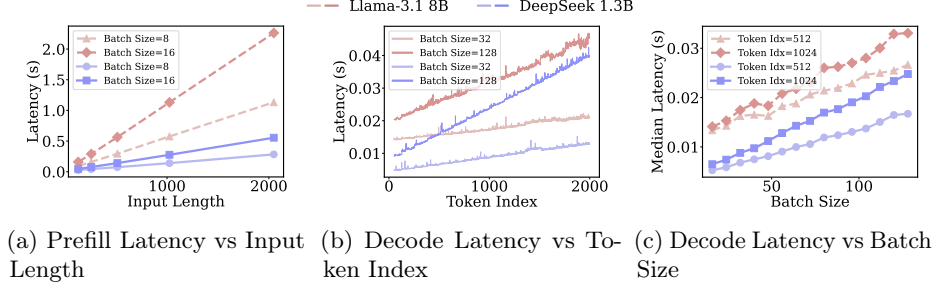


Fig. 2: Impact of input sequence length (a), batch size (b), and output token index (c) on prefill latency and inter-token latency (ITL).

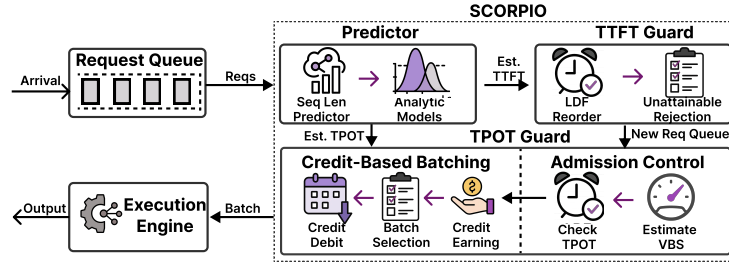


Fig. 3: The workflow of SCORPIO.

the TTFT Guard (§3.3) first reorders the requests according to their TTFT SLO deadline (i.e., Least Deadline First Reordering). Additionally, it uses the estimated TTFT to reject requests that are unattainable with respect to their TTFT SLOs. Given the new reordered requests, the TPOT Guard (§3.4) uses the estimated TPOT to decide whether to admit the requests into the running batch (i.e., VBS-based Admission Control Mechanism). Then, it employs a Credit-based Batching Mechanism to select which requests to batch in each processing iteration. Lastly, the selected requests are batched and processed in the execution engine.

3.2 Predictor

Sequence Length Predictor. We design a lightweight predictor for output sequence length with minimal overhead. In contrast to prior work [8,21], which typically uses coarse-grained bucketing (e.g., 10 bins), we conduct a systematic analysis of bucketing strategies and show that medium-grained binning (e.g., 100 bins) provides superior accuracy: for Llama3-8B on ShareGPT, 100 equal-width bins give Kendall’s τ of 0.54 versus 0.25 at 10 bins. This design improves on prior approaches along two axes: (i) it corrects the class imbalance that inflates overall accuracy while degrading prediction on minority classes, and (ii) it sustains

resolution as maximum context lengths grow in modern LLMs. We fine-tune OPT-125M [32] as a length-bin classifier on 20K ShareGPT and LMSYS-Chat-1M samples and co-locate it with the serving engine.

SLO Metric Estimation. We observe two key statistical correlations from our empirical analysis (Figure 2, profiled on a representative model as these trends are consistent across models): (1) inter-token latency (ITL) is positively correlated with batch size and average sequence length, and (2) prefill time is positively correlated with prompt length. Based on these observations, we develop analytical models to estimate TPOT and TTFT for scheduling decisions.

For TPOT estimation, we model ITL as $\mathcal{F}(|R|, L)$ where $|R|$ is batch size and L is average sequence length. Given a new request r with predicted output length $P(r)$, we estimate the batch-level TPOT as the expected ITL over the next $P(r)$ generation steps. For TTFT estimation, we model prefill time as $\mathcal{G}(L_P)$ where L_P is prompt length. The estimated TTFT for a waiting request is the cumulative prefill time of all preceding requests in the queue. Across both models and datasets, these estimators attain R^2 above 0.9 (0.90 to 0.99).

3.3 Heterogeneous TTFT Guard

Least Deadline First (LDF) Reordering. To handle heterogeneous TTFT SLOs, SCORPIO uses a simple but effective LDF reordering strategy. The deadline for a request r_i is calculated as the time left to its TTFT deadline $S_{TT}(r_i)$. This strategy puts more urgent requests (with earlier TTFT deadlines) at the front of the queue, achieving better system-level TTFT attainment.

Unattainable SLO Rejection. When the system load is excessively high, some requests will inevitably violate their TTFT SLO and need special handling. In this paper, we tag these requests as unattainable SLO requests and reject them for simplicity. These rejected requests integrate naturally with complementary cloud-side mechanisms such as cross-node migration or elastic scaling [28], making SCORPIO a drop-in scheduling layer within a larger serving fabric.

Starvation Prevention. The combination of LDF Reordering and Unattainable SLO Rejection inherently prevents request starvation. As time progresses, requests with approaching deadlines gain higher priority and move to the front of the queue, while requests that would inevitably miss their deadlines are proactively rejected rather than allowed to starve.

To quantitatively assess starvation prevention in our heterogeneous SLO environment, we propose a *max_waiting_ratio* fairness metric to evaluate the fairness among requests. Different from absolute waiting time metrics [5], we define *max_waiting_ratio* as:

$$max_waiting_ratio = \max_{r \in R} \left(\frac{waiting_time(r)}{S_{TT}(r)} \right) \quad (4)$$

where $S_{TT}(r)$ is the TTFT SLO requirement for request r . This metric captures relative starvation by accounting for individual SLO, where lower values indicate better starvation prevention (§4.4).

3.4 Heterogeneous TPOT Guard

Key Insight. Treating every request identically wastes the system’s most valuable lever: requests with looser TPOT can be batched less frequently, returning compute to requests with tighter TPOT and converting per-request SLO slack into system-wide guarantee capacity. As shown in Figure 1, existing methods indiscriminately admit all incoming requests and process them equally in each step. This approach leads to two primary issues: 1) SLO violations for requests with strict TPOTs when contending with those having looser TPOT SLOs, and 2) when the system workload exceeds its processing capacity, greedily serving all requests causes a cascading effect where all requests fail to meet their SLOs. To address the first issue, we design a novel batching mechanism that provides differentiated batching opportunities based on a request’s TPOT SLO (i.e., Credit-based Batching Mechanism). Requests with looser TPOT SLOs are offered fewer credits (opportunities) for batching, thereby skipping some execution iterations and leaving more resources for requests with tighter TPOT SLOs. For the latter issue, inspired by load control techniques in cloud computing [28], we introduce a novel admission control mechanism that accounts for the heterogeneity of TPOT SLOs. On one hand, requests estimated to cause TPOT violations are rejected. On the other hand, since a request with looser TPOT SLOs is served intermittently, it can be regarded as a partial request when calculating the batch size (i.e., Virtual Batch Size) for TPOT estimation (§3.2). These two mechanisms together enforce TPOT SLOs in a best-effort manner, as illustrated in Algorithm 1.

Credit-based Batching offers requests of different TPOT SLOs with different credits (opportunities) for batching each iteration as mentioned above. To decide how many credits a request earns, we first introduce a concept called TPOT-relative Proportionality (TRP):

Definition 1 (TPOT-relative Proportionality (TRP)). Let $S_{TP}(r)$ denote the TPOT SLO of request r . Given current processing iteration t and the running requests set $R(t)$, the TPOT-relative Proportionality (TRP) of a request $r \in R(t)$ is defined as:

$$TRP(r) = \frac{\min_{r' \in R(t)} S_{TP}(r')}{S_{TP}(r)}$$

The TRP quantifies the urgency of a request r to be batched compared to admitted requests with the strictest TPOT SLOs. Note that the TRP of a request adaptively responds to changing workloads. Therefore, each request earns credits at its TRP. Accumulating sufficient credit (≥ 1.0) grants a request to be batched in the next processing batch. Its credit is then decremented by 1.0, representing the consumption of one processing opportunity. Formally, in each batching step t , the following actions are taken:

- **Credit Earning:** For every request $r \in R(t)$, its credit is updated based on its TRP rate:

$$C_r(t) \leftarrow C_r(t) + TRP(r)$$

where $C_r(t)$ is the credit of request r at step t , which is initialized as 0.

- **Batch Selection:** The batch $\mathcal{B}(t)$ is formed by including all requests whose credit, after accumulation, is greater than or equal to the threshold:

$$\mathcal{B}(t) = \{r \in R(t) \mid C_r(t) \geq 1.0\}$$

- **Credit Debit:** For every request r included in $\mathcal{B}(t)$, its credit is decremented by 1.0:

$$\text{If } r \in \mathcal{B}(t), \text{ then } C_r(t) \leftarrow C_r(t) - 1.0$$

This mechanism ensures that over many steps, the frequency of a request r being batched will converge towards its TRP rate.

VBS-based Admission Control. When a new request r arrives, to guarantee TPOT SLOs, an intuitive approach is to admit requests into the running batch if admitting it will neither violate its own TPOT SLO nor cause other running requests to violate their TPOT SLOs. However, since credit-based batching causes some requests to skip some execution iterations as mentioned above, directly using the number of running requests as batch size overestimates the actual system load. Since the request r can earn a $TRP(r)$ opportunity to be batched in each iteration if admitted, it can be regarded as a virtual $TRP(r)$ request. Let $R(t)' = R(t) \cup \{r\}$, we can project the actual load of the system as the sum of the TRP of all requests, which we denote as virtual batch size (VBS):

$$\text{VBS}(R') = \sum_{r \in R'} TRP(r) \quad (5)$$

The request r is admitted to the running queue at step t if adding it would not cause the estimated TPOT to exceed the minimum TPOT SLO of the running requests set R' :

$$\text{EstimatedTPOT}(\text{VBS}(R'), L_{avg}(R')) \leq \min_{r' \in R'} S_{TP}(r')$$

The mechanism drives the admitted requests toward their TPOT SLOs. Because the predictor and analytic models are imperfect, SCORPIO targets high statistical attainment rather than hard guarantees, and rejecting requests estimated to violate their TPOT SLOs bounds the impact of mispredictions.

4 Experiments

We evaluate SCORPIO against state-of-the-art baselines and analyze the contribution of each component.

4.1 Experimental Setup

Testbed. We use a server with 4 NVIDIA A100 GPUs (80GB each), connected by NVLink within pairs (GPU0-GPU1, GPU2-GPU3) and PCIe/NUMA across pairs.

Algorithm 1: TPOT Guarantee Mechanism

Input: LLM model M , Sequence Length Predictor P , LDF Sorted Waiting Queue W , Running Queue R

Output: Batched requests set B

```

1 while True do
2    $B \leftarrow \emptyset$ 
3   foreach  $w \in W$  do
4      $R' \leftarrow R \cup \{w\}$ 
5     if  $EstimatedTPOT(VBS(R'), L_{avg}(R')) \leq \min_{r \in R'} S_{TP}(r)$  then
6        $B \leftarrow B \cup \{w\}$ ,  $W \leftarrow W \setminus \{w\}$ ,  $R \leftarrow R'$  // Admission Control
7     end
8   end
9   foreach  $r \in R$  do
10     $C_r(t) \leftarrow C_r(t) + TRP(r)$  // Credit Earning
11    if  $C_r(t) \geq 1.0$  then
12       $B \leftarrow B \cup \{r\}$ ,  $C_r(t) \leftarrow C_r(t) - 1.0$  // Batch Selection and Credit Debit
13    end
14  end
15 end

```

Serving Models. We use Meta Llama-3.1 8B [6] and Google Gemma-2 27b [26] as serving models, in FP16/BF16 precision. The 8B model runs on a single GPU, and the 27B model runs on 4 GPUs with tensor parallelism [25].

Workloads. We evaluate on the ShareGPT [27] and LMSYS-Chat-1M [33] datasets, widely used collections of real-world conversations. To stress heterogeneous serving, following prior works [12,35] we consider 6 SLO categories, summarized in Table 1: Category 1 has tight TTFT and TPOT (e.g., code generation); Categories 2-3 have tight TPOT and looser TTFT (e.g., tool calls); Categories 4-5 have loose TPOT and tight TTFT (e.g., reading-speed chatbots); Category 6 is loose on both (e.g., summarization). For the 27B model we loosen the constraints to account for the larger model size.

Table 1: SLO categories for different model sizes.

Model	Metric	Category					
		1	2	3	4	5	6
Llama-3.1 8B	TTFT (s)	0.5	2	3	0.5	1	7.5
	TPOT (ms)	30	30	30	50	50	50
Gemma-2 27B	TTFT (s)	1.0	4	6	1.0	2	15
	TPOT (ms)	60	60	60	100	100	100

Baselines. We compare our method with the following baselines:

- **vLLM** [9]: A state-of-the-art LLM serving system that uses a throughput-oriented scheduling strategy. The vLLM scheduler prioritizes prefills.
- **S3** [8]: An LLM inference system that predicts the output sequence length and employs the shortest job first scheduling. Since S3 is implemented on basic LLM inference without techniques like PagedAttention, we re-implement its core scheduling strategy within vLLM for a fair comparison.
- **Mooncake** [20]: A production-grade LLM serving platform that employs an early-rejection-based admission control for SLO guarantees. We integrate this mechanism into vLLM.

4.2 QPS-Scaling SLO Attainment

We compare SCORPIO’s goodput and SLO attainment rate against baselines on ShareGPT and LMSYS-Chat-1M with heterogeneous SLOs, under varying request arrival rates [12,35]. As shown in Figure 4, SCORPIO achieves higher goodput and SLO adherence than the baselines, especially at high QPS. For example, at a QPS of 15, it yields up to 8.8-14.4 $\times$ higher goodput and 40.7-46.5% higher SLO adherence than baselines, demonstrating robust burst handling through its SLO-oriented scheduling. In contrast, vLLM’s greedy admission leads to severe TPOT violations. S3’s output-length ranking and Mooncake’s strict rejection both exhibit suboptimal SLO adherence. At low load the system is not the bottleneck and all methods meet most SLOs, so the small overhead of co-locating the predictor (quantified in §4.4) slightly reduces SCORPIO’s goodput in the regime where scheduling matters least. On long-input and long-output workloads (1K-2K tokens), SCORPIO sustains consistent gains over baselines, in line with the main results.

4.3 Real-World Trace Serving

We further evaluate SCORPIO using 20-minute real-world traces [2]; the 20-min Azure trace mixes bursty and light load, peaking above 60 req/s. Figure 5 reports the cumulative number of SLO-met requests for Llama3-8B and Gemma2-27B models across both datasets. Across all settings, SCORPIO achieves the highest cumulative SLO-met counts. For example, SCORPIO achieves a 1.25, 2.01, and 2.11 $\times$ higher SLO-met requests compared with Mooncake, vLLM, and S3, respectively. Throughput-oriented baselines like vLLM exhibit noticeable slowdowns during traffic spikes (e.g., minutes 3-5 and 9-11), caused by uniform batching and greedy admission, leading to widespread TPOT violations. Mooncake (Figure 5b,d) alleviates some pressure via early rejections, but its SLO gains taper off with a too strict admission control mechanism and lack of consideration for the heterogeneous SLOs. S3’s length ranking also exhibits suboptimal SLO attainment. In contrast, SCORPIO steadily maintains a lead in cumulative SLO-met requests. By combining TTFT Guard and TPOT Guard, SCORPIO effectively handles the heterogeneous SLOs.

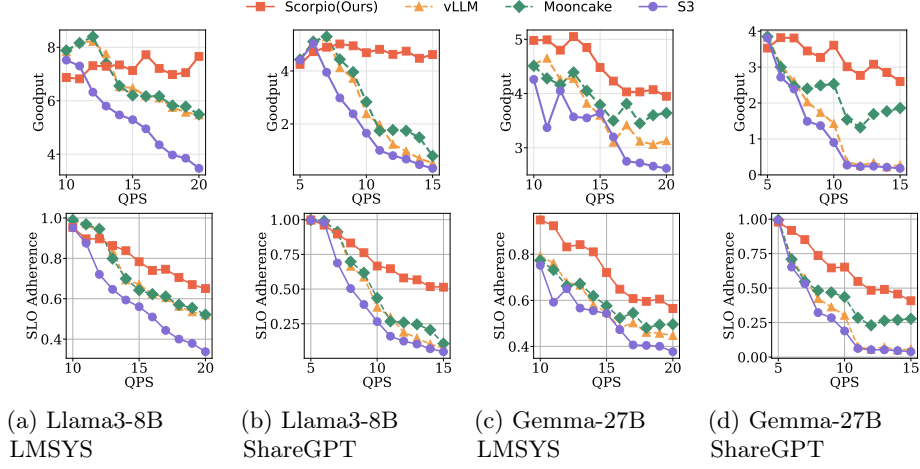


Fig. 4: Impact of different scheduling strategies on goodput and SLO adherence vs QPS.

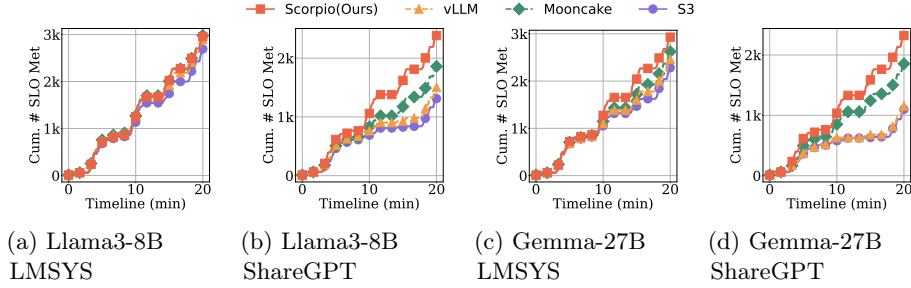


Fig. 5: Cumulative number of SLO-met requests over time on the LMSYS and ShareGPT dataset.

4.4 Effectiveness Analysis

Ablation Study. For this evaluation, we evaluate the core components of SCORPIO. As shown in Figure 7, we add the proposed components incrementally and assess the performance at a representative high-load QPS of 14. We find that 1) without any component, SCORPIO’s goodput is significantly reduced. 2) Including only the TTFT Guard effectively reduces TTFT violations as intended, but results in severe TPOT violations, and vice versa. These results show the interdependence of the components, highlighting their importance for the overall performance.

Overhead of SCORPIO’s Scheduling. Summing the time of all scheduling sub-components (TTFT Guard and TPOT Guard) when responding to 512 requests, SCORPIO adds under 1% scheduling overhead in all four model-dataset settings

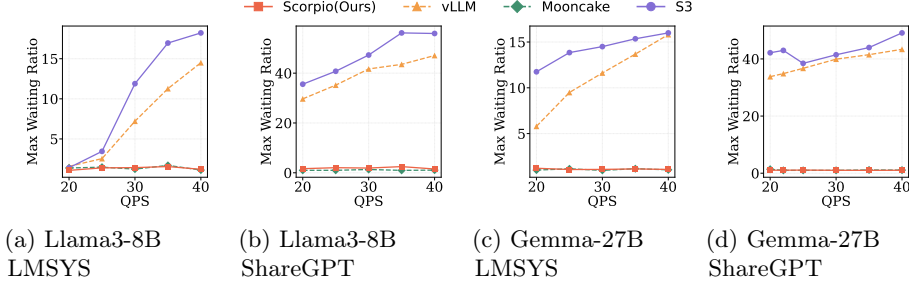


Fig. 6: Starvation prevention on the impact of different scheduling strategies.

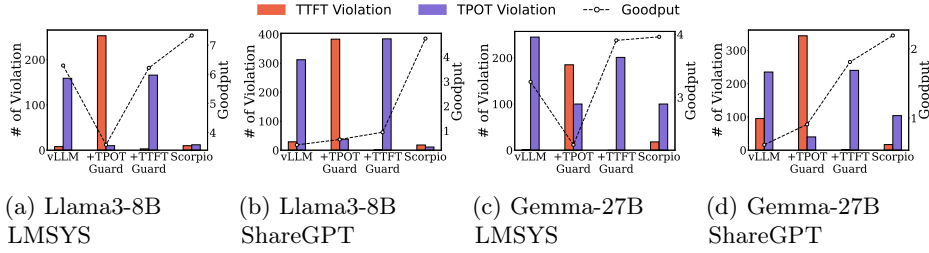


Fig. 7: Ablation study on the impact of different scheduling strategies.

(0.11% to 0.17%). We do not account the time of the predictor since it is shown to be negligible in previous works [8,5].

Starvation Prevention. We evaluate the starvation prevention of SCORPIO, shown in Figure 6. SCORPIO achieves comparable mean *max_waiting_ratio* with Mooncake, while outperforming other baselines by 1-2 orders of magnitude. These results confirm that SCORPIO successfully prevents request starvation in heterogeneous SLO environments.

5 Related Work

Throughput-oriented serving. A first line of work pursues high serving throughput by improving GPU utilization. Orca [30] and vLLM [9] introduce continuous batching and paged attention, which have become de facto building blocks for modern serving stacks [11] and production engines such as SGLang [34] and TensorRT-LLM [17]. Building on these foundations, Sarathi-Serve [1] piggybacks decode steps onto chunked prefills to keep the accelerator busy. A complementary direction disaggregates the prefill and decode phases onto separate instances: SplitWise [18] and DistServe [35] show that isolating compute-bound prefill from latency-sensitive decode removes mutual interference. SCORPIO builds on this throughput-oriented foundation and extends it toward an explicitly SLO-aware objective.

SLO- and latency-aware scheduling. As LLMs increasingly back Web services with diverse latency expectations, recent work shifts attention from raw throughput to Service Level Objectives. Classic inference-serving systems already framed scheduling around SLO guarantees and cost across heterogeneous hardware [13,31]. In the LLM setting, Mooncake [20] adopts a KVCache-centric architecture with a prediction-based early-rejection policy for overload, while QLM [19] estimates request waiting times to orchestrate queue management for TTFT adherence. Fairness-oriented schedulers further ensure equitable service across clients [24,23]. Closest to our setting, AdaServe [12] and SLOs-Serve [3] adjust per-request token allocation on the fly, the latter combining chunked prefill with optional speculative decoding [10]. We explore a new setting in which requests carry *heterogeneous* TTFT and TPOT targets simultaneously, and design fine-grained guards that exploit this heterogeneity rather than treating SLOs uniformly.

Length prediction for scheduling. Because output length is unknown a priori, a body of work predicts request characteristics to inform scheduling. Length-aware policies enable shortest-job-first style ordering, via direct length estimation [8,4,21], learning-to-rank over batches [5], or embedding-based shortest-remaining-time scheduling [22]. Prediction has also been applied at the LMaaS layer for request management and routing [7]. SCORPIO uses a lightweight length predictor as one input to an analytic SLO-estimation module, turning predictions into admission-control and batch-selection decisions under heterogeneous SLOs.

6 Conclusion

In this paper, we present SCORPIO, a novel SLO-oriented LLM serving system designed to maximize system goodput and SLO adherence for requests with heterogeneous SLOs. By turning SLO heterogeneity from a scheduling burden into a source of capacity, SCORPIO coordinates a TPOT Guard and a TTFT Guard, supported by an accurate predictor, to serve the right request at the right time. Evaluations demonstrate that SCORPIO significantly outperforms state-of-the-art throughput-oriented systems, improving system goodput by up to $14.4\times$ and SLO adherence rate by up to 46.5% across various scenarios. SCORPIO integrates cleanly as a scheduling layer atop existing serving stacks and complements optimizations such as prefill-decode disaggregation [35].

Acknowledgments. This work was supported in part by the National Natural Science Foundation of China (62132017, 62502430), and Zhejiang Provincial Natural Science Foundation of China (LQ26F020004).

References

1. Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B.S., Ramjee, R.: Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills (2023), <https://arxiv.org/abs/2308.16369>

2. Azure, M.: Azuretrace. <https://github.com/Azure/AzurePublicDataset> (2023), accessed: 2023
3. Chen, S., Jia, Z., Khan, S., Krishnamurthy, A., Gibbons, P.B.: SLOs-Serve: Optimized Serving of Multi-SLO LLMs (Apr 2025). <https://doi.org/10.48550/arXiv.2504.08784>
4. Cheng, K., Hu, W., Wang, Z., Du, P., Li, J., Zhang, S.: Enabling efficient batch serving for lmaas via generation length prediction (2024), <https://arxiv.org/abs/2406.04785>
5. Fu, Y., Zhu, S., Su, R., Qiao, A., Stoica, I., Zhang, H.: Efficient LLM Scheduling by Learning to Rank (Aug 2024). <https://doi.org/10.48550/arXiv.2408.15792>
6. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., et al: The llama 3 herd of models (2024), <https://arxiv.org/abs/2407.21783>
7. Jiang, Z., Huang, Y., Yu, G., Huang, J., Gu, J., Lyu, M.R.: Hierarchical prediction-based management for lmaas systems (2025), <https://arxiv.org/abs/2504.03702>
8. Jin, Y., Wu, C.F., Brooks, D., Wei, G.Y.: S^3 : Increasing GPU Utilization during Generative Inference for Higher Throughput (Jun 2023). <https://doi.org/10.48550/arXiv.2306.06000>
9. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient Memory Management for Large Language Model Serving with PagedAttention (Sep 2023). <https://doi.org/10.48550/arXiv.2309.06180>
10. Leviathan, Y., Kalman, M., Matias, Y.: Fast inference from transformers via speculative decoding (2023), <https://arxiv.org/abs/2211.17192>
11. Li, B., Jiang, Y., Gadepally, V., Tiwari, D.: LLM Inference Serving: Survey of Recent Advances and Opportunities (Jul 2024). <https://doi.org/10.48550/arXiv.2407.12391>
12. Li, Z., Chen, Z., Delacourt, R., Oliaro, G., Wang, Z., Chen, Q., Lin, S., Yang, A., Zhang, Z., Chen, Z., Lai, S., Miao, X., Jia, Z.: AdaServe: SLO-Customized LLM Serving with Fine-Grained Speculative Decoding (Jan 2025). <https://doi.org/10.48550/arXiv.2501.12162>
13. Liu, G., Shen, H., Wang, H.: An economical and slo-guaranteed cloud storage service across multiple cloud service providers. *IEEE Transactions on Parallel and Distributed Systems* **28**(9), 2440–2453 (2017)
14. Manus: Manus (2025), <https://manus.im/>
15. Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Jin, H., Chen, T., Jia, Z.: Towards efficient generative large language model serving: A survey from algorithms to systems (2023), <https://arxiv.org/abs/2312.15234>
16. Nguyen, N., Nadi, S.: An empirical evaluation of github copilot’s code suggestions. In: *Proceedings of the 19th International Conference on Mining Software Repositories*. pp. 1–5 (2022)
17. NVIDIA: TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM> (2024), accessed: 2024
18. Patel, P., Choukse, E., Zhang, C., Shah, A., Íñigo Goiri, Maleki, S., Bianchini, R.: Splitwise: Efficient generative llm inference using phase splitting (2024), <https://arxiv.org/abs/2311.18677>
19. Patke, A., Reddy, D., Jha, S., Qiu, H., Pinto, C., Narayanaswami, C., Kalbarczyk, Z., Iyer, R.: Queue management for slo-oriented large language model serving (Feb 2025). <https://doi.org/10.1145/3698038.369852>
20. Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., Xu, X.: Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving (Jul 2024). <https://doi.org/10.48550/arXiv.2407.00079>

21. Qiu, H., Mao, W., Patke, A., Cui, S., Jha, S., Wang, C., Franke, H., Kalbarczyk, Z.T., Başar, T., Iyer, R.K.: Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction (Nov 2024). <https://doi.org/10.48550/arXiv.2404.08509>
22. Shahout, R., Malach, E., Liu, C., Jiang, W., Yu, M., Mitzenmacher, M.: Don't stop me now: Embedding based scheduling for llms (2024), <https://arxiv.org/abs/2410.01035>
23. Shen, A., Li, Z., Gao, M.: Fastswitch: Optimizing context switching efficiency in fairness-aware large language model serving. arXiv preprint arXiv:2411.18424 (2024)
24. Sheng, Y., Cao, S., Li, D., Zhu, B., Li, Z., Zhuo, D., Gonzalez, J.E., Stoica, I.: Fairness in serving large language models. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 965–988 (2024)
25. Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B.: Megatron-lm: Training multi-billion parameter language models using model parallelism (2020), <https://arxiv.org/abs/1909.08053>
26. Team, G.: Gemma 2: Improving open language models at a practical size (2024), <https://arxiv.org/abs/2408.00118>
27. Team, S.: Sharegpt (2023), <https://sharegpt.com/>
28. Wang, Q., Lan, T., Tang, Y., Huang, Z., Du, Y., Zhang, H., Sha, J., Lu, H., Zhou, Y., Zhang, K., Tang, M.: Dlover-rm: Resource optimization for deep recommendation models training in the cloud (2024), <https://arxiv.org/abs/2304.01468>
29. xAI: Grok-3 (2025), <https://x.ai/news/grok-3>
30. Yu, G.I., Jeong, J.S., Kim, G.W., Chun, B.G.: Orca: A Distributed Serving System for Transformer-Based Generative Models
31. Zhang, C., Yu, M., Wang, W., Yan, F.: {MArk}: Exploiting cloud services for {Cost-Effective},{SLO-Aware} machine learning inference serving. In: 2019 USENIX Annual Technical Conference (USENIX ATC 19). pp. 1049–1062 (2019)
32. Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X.V., Mihaylov, T., Ott, M., Shleifer, S., Shuster, K., Simig, D., Koura, P.S., Sridhar, A., Wang, T., Zettlemoyer, L.: Opt: Open pre-trained transformer language models (2022), <https://arxiv.org/abs/2205.01068>
33. Zheng, L., Chiang, W.L., Sheng, Y., Li, T., Zhuang, S., Wu, Z., Zhuang, Y., Li, Z., Lin, Z., Xing, E.P., Gonzalez, J.E., Stoica, I., Zhang, H.: Lmsys-chat-1m: A large-scale real-world llm conversation dataset (2024), <https://arxiv.org/abs/2309.11998>
34. Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C.H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J.E., Barrett, C., Sheng, Y.: SGLang: Efficient execution of structured language model programs. In: The Thirty-eighth Annual Conference on Neural Information Processing Systems (2024), <https://openreview.net/forum?id=VqkAKQibpq>
35. Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H.: Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving (2024), <https://arxiv.org/abs/2401.09670>